

Management for Distributed Computing Environment-based applications

N Winton

Many applications that work well in a development environment may fail in a live operational environment because they can not be monitored and managed. The potential problems in a distributed client/server environment are even greater. This paper describes some of the main issues and the environment within BT into which new applications must fit. It then discusses the innovative work done to address the problems of providing a comprehensive and extensible solution for BT's Distributed Computing Environment-based applications.

1. Introduction

A distributed system is one in which your file fails to print because a computer of which you have never heard has just crashed'. This definition, attributed to Leslie Lamport, may be pessimistic but does capture one of the less desirable aspects of modern client/server and distributed systems. The likelihood of a service-affecting failure occurring increases as more machines and network links are introduced.

Given such an outlook it may be questioned as to why anyone should want to build and use distributed systems. The answer for BT, as with many other companies, is that there is no alternative. The need to be ever more responsive to customer demands and to support increasingly complex services means that processes must be distributed across a range of existing operational systems.

BT has a history of successfully building and operating some of the world's largest and most complex non-distributed applications. As a future in which distributed computing becomes common moves ever closer, the least that must be provided is the same level of service and reliability that users have come to expect.

2. The main issues

Many applications that work well in a development environment fail in a 'lights-out' operational environment because they cannot be monitored and managed. A key aspect of building successful applications — distributed or not — is to ensure not only that they function to specification but that operations personnel have sufficient information and control to keep them running smoothly.

The following sections describe briefly some of the main challenges facing operations and, hence, application developers.

2.1 End-to-end management

Users do not and should not care whether the applications on their desktops are distributed or not. They do care, however, whether those applications are working or not. It is very little comfort to an irate user to be told that 'the server appears to be working perfectly' if a connecting network link has been lost.

Administrators need to see the 'big picture.' Every component of a distributed system needs to be monitored and managed. Not only that, but many different sources of information must be integrated, correlated and presented to allow the root cause of a problem to be determined quickly and accurately.

2.2 Complexity and diversity

Client/server applications have more to go wrong. Problem diagnosis is hindered by this additional complexity and compounded by unnecessary diversity. It cannot be expected that every administrator is an expert in every possible software and network technology. Therefore the management of applications needs to be made as consistent as possible.

2.3 *Reactive and proactive management*

There is a desire to operate both reactively — responding to exceptional conditions raised by an application — and proactively — monitoring to predict and eliminate failures before they happen. This requires that applications are properly instrumented to provide the necessary information. To meet the needs already outlined it also implies that there should be a consistent mechanism by which the information can be collected.

2.4 *Security and delegation*

The management regime must be secure but also permit delegation of responsibility for certain actions where appropriate. It is impossible for any single central team to have the knowledge and resources to manage all of BT's systems and applications completely. Further, when an application is distributed across machines in different geographic locations some level of local support for host systems is inevitable.

A structured hierarchy of expertise and responsibility is therefore required.

3. **The BT environment**

The issues outlined above are by no means unique to BT. However, the company does have specific organisational and technology structures into which applications must fit.

The applications with which this paper is concerned run on servers in a number of geographically dispersed data centres. Clients may be deployed in any of several hundred buildings. Machines share access to a high-speed, company-wide multi-protocol network. The network is a mixture of Ethernet and Token Ring LANs, FDDI MANs and Frame Relay WANs. Most systems have access via at least 2 Mbit/s links.

Server machines are centrally managed from a small number of 'command centres' distributed across the UK. These centres look after the basic health of networks, machines and software. They monitor events and route application-specific problems to the appropriate application support groups. To do this they need sufficient information to be able to decide on the correct path for problem resolution but they do not want to be deluged with detailed information. So, for example, they need to determine whether an apparent application fault is in fact caused by underlying network problems.

The application support groups have more detailed knowledge of particular software. They require more specific information and 'hands-on' access to application data.

3.1 *Tools*

A standard set of tools is used in order to assist in monitoring and managing, with particular regard to open systems platforms. These tools together form the enterprise systems management framework (ESMF). The core monitoring tool is based on IBM's Netview/6000. This uses the Internet Simple Network Management Protocol (SNMP) [1]. Another IBM product based around their system management interface tool (SMIT) and a remote invocation mechanism called XSA is used to perform remote operations.

SNMP supports three basic types of operations — get attribute values, set attribute values and 'traps.' A trap is essentially an asynchronous event report. SNMP version 1, currently being used, is notoriously insecure, so only 'get' operations and traps are used within ESMF. No set operations are used to alter the state of managed systems.

Hosts managed via SNMP generally run an SNMP agent process. This receives incoming requests for operations on its management information base (MIB). This MIB provides a logical view of the managed resources within the machine. Operations on the MIB usually result in some corresponding operation on the underlying system.

A simple command (bttrap) is also provided on all machines to allow them to generate traps. This takes care of constructing an SNMP trap message and delivering it to management stations.

XSA essentially allows remote command scripts to be scheduled and run from a central machine. Again, each machine runs an agent which processes requests.

4. **The challenge**

BT has chosen the Distributed Computing Environment (DCE), originally produced by the Open Software Foundation, as one of the key technologies for building new client/server systems. DCE provides a set of tools for building secure, reliable and scalable distributed applications. These include:

- a remote procedure call (RPC) mechanism,
- local and global directory services,
- integrated security system,
- time synchronisation services.

While industry has worked to address the management of the core DCE facilities themselves, there has been less work on the management of applications built on DCE. Specialised environments such as the Encina transaction processing environment from Transarc or the Entera tool-set from Open Environment Corporation go some way to

addressing this. However, no tool offered immediate integration into the BT environment or the full level of control or monitoring required.

The primary focus was in building manageable DCE servers. The challenge was to provide a generic mechanism by which servers could be integrated with ESMF with the least effort but which still gave the flexibility to support a wide range of application-specific management requirements.

5. The solution

In some respects the ESMF technology choices of SNMP and XSA could be seen as a backwards step in the DCE environment. DCE provides a much more general, flexible and secure mechanism for performing remote operations than either of these. The main goal was to fit cleanly into the ESMF infrastructure by finding ways to work with SNMP and XSA. However, there was no desire to burden application developers with having to learn and program SNMP and XSA.

5.1 The management interface

The approach taken was to define a standard DCE interface which all managed servers must support in addition to their 'normal' functional interface. Having defined this interface, server developers are isolated from the details of integrating with SNMP and XSA. It allows clients to be built which can perform the translation between SNMP, XSA or any other management regime.

The management interface provides a basic set of operations. Some of these apply to the server as a whole, and others that are performed on managed 'components' of the server. The operations on the server are as follows:

- stop the server,¹
- back-up and restore server state,
- perform server-specific tests,
- list the managed components.

For each managed component that the server supports there are a further set of operations, as follows:

- disable and enable the component,
- save and load component state,
- perform component-specific tests,
- list attributes of the component,
- get an attribute value,

¹ There is, obviously, no 'start-server' operation as it is hard to ask the server to perform an operation if it is not already available to field the request. Server start-up is handled through a separate mechanism.

- set an attribute value.

DCE itself provides some rudimentary management functions — shut-down, liveness test and basic statistics — for all servers. However, it was felt that there was a need for a more refined degree of monitoring and control. For example, the stop operation, in addition to supporting an immediate forced shut-down, also provides for a more graceful termination allowing components to be quiesced before the server is stopped. Similarly, there was also a need to support an orderly 'in-flight' back-up mechanism so the back-up/restore and save/load operations can be used to ensure that key information is written to disk before a normal machine back-up is performed.

The 'get' and 'set' operations correspond closely to the SNMP model of management. The interface is designed to be generic and to allow dynamic discovery of both the components and attributes it supports. This means that a client does not have to have any prior knowledge of a MIB for the server — although in many cases this is, of course, desirable.

The basic attribute types supported for components were modelled closely on those in SNMP. All of the core SNMP types defined in Rose and McCloghrie [2] were represented. This was done to make the mapping through to an SNMP MIB definition as simple as possible. With hindsight, this was probably unnecessary and caused confusion to developers who were unclear, for example, of the distinction between an SNMP 'counter' and a 'gauge'. In practice a set of types limited to signed and unsigned 32-bit integers, printable character strings and opaque binary data would probably have sufficed and have been easier for most programmers to understand.

The interface also inherited from SNMP the concept of tables of values. Each column of a table is limited to being one of the primitive data types but a table can have infinitely many columns and rows. It would have been possible to have allowed richer data-structuring mechanisms but for the desire to have data representable in SNMP.

One way in which the data-typing does not follow SNMP is in attribute naming. SNMP attributes are identified using ASN.1 object identifiers (OIDs). Component attributes in the management interface are identified using simple, human-readable, textual names.

Although all the attribute names and types can be discovered dynamically through the management interface, a core set of attributes has also being defined for each component. So, for example, there is an attribute to indicate whether a particular component is currently enabled or disabled. Another important set of attributes provide fine-grained performance and utilisation statistics.

There is no facility within the management interface itself to generate out-bound 'traps' or other alerts. This is a deliberate feature of the design. While it would be possible for a server to make an outbound RPC itself (i.e. acting as a client) the existing SNMP trap mechanism (via the `btrap` command) was thought to be more than sufficient for immediate needs.

5.2 The server infrastructure

Although the management interface is relatively straightforward it would be onerous and error prone to force every server developer to re-create the code to implement it. Effort was therefore invested in building C++ classes that provide server developers with most of the management functionality 'for free'.

The server infrastructure contains a number of components (some of which are themselves directly manageable through the management interface). The code devoted to implementing the management functions themselves amounts to approximately 10 000 lines of C++. This code handles all of the business of initialising the server and reading configuration files. It also provides default behaviour (which can be overridden if necessary) for all of the management interface functions. This includes supporting the default set of attributes for components.

Configuration information and other persistent management data are stored in normal text files. While a more sophisticated database system may have simplified development, there was no wish to impose the use of an RDBMS on those systems which did not otherwise need one. Text files also have the advantage that they are easy to create initially and, perhaps more importantly, they can be fixed by the commonest systems management tool — a text editor.

As already mentioned above, the infrastructure makes use of the existing `btrap` command to generate SNMP traps. A single, simple logging programming interface is provided, however, so that the decision as to whether a particular event should generate a trap or just log a message is configurable.

5.3 The management client

Having each server support the management interface would be of little value unless there were something that could use it. It would be possible to write specific client programs to manage different servers. However, it was felt that a much better approach would be to write a single generic client which could easily be extended or customised. It should be possible to configure the client to meet the needs of both specific operational policies and server implementations.

Such a client needs to provide access to the complete range of functionality of the management interface. It should be possible to manipulate all valid data-types in a consistent manner. These goals led to the belief that the management client should provide some form of extension language or scripting capability. Certainly the management client should have the ability to be used directly by human users. Configurability and flexibility is useful in interactive use. However, there was another motivation for wanting a configurable client program.

The idea of writing scripts to perform management functions on a server fits well with the mechanisms available for integrating with the overall management infrastructure. Both the SNMP and XSA agents in use on BT's machines were already designed to perform local operations by invoking appropriate local programs or scripts. It was, therefore, very useful to have the capability to produce and tailor these scripts quickly. The positioning of the management client is shown in Fig 1.

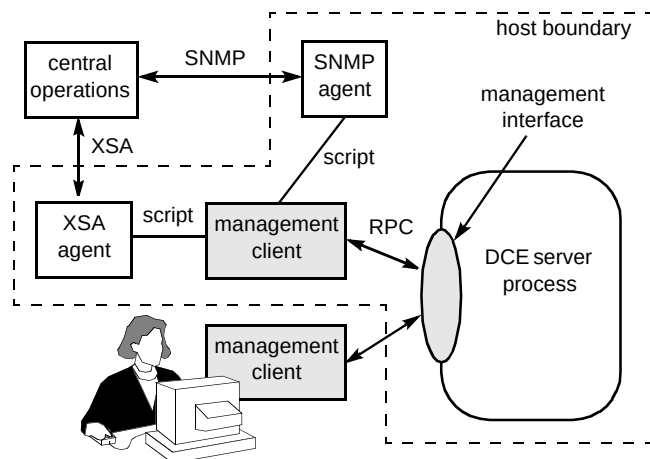


Fig 1 Positioning of the management client.

5.4 Use of Tcl

The Tcl system [3] was chosen to build the generic management client. Tcl (tool command language — pronounced 'tickle') is designed to be a simple, easily-extensible scripting language. It provides a set of basic programming capabilities (control structures, procedures, etc) as well as a large number of string, list and file manipulation facilities. The syntax of Tcl is reminiscent of other Unix shell command languages.

Tcl is freely available (although its primary author, John Ousterhout, is now employed by Sun where development continues). It has become quite widely used within the Internet community.

A primary aim of Tcl was to provide language that could be easily extended. It is possible to add new

commands to the language by writing fairly straightforward C language routines and linking with the Tcl library. Another major motivation for the adoption of Tcl was its use in the DCE 1.1 'DCE Control Program' (dcecp). This is the primary administrative tool for DCE 1.1 and so familiarity with Tcl acquired for this purpose would be equally applicable to the management client.

5.5 Extending Tcl

In order to add the required functionality to the Tcl core a single command was added (sysman) with sub-commands reflecting each of the functions provided in the management interface. Sub-commands were also provided to connect to and disconnect from a specified server ² and to log into or out of the DCE environment.

A library of additional routines useful for scripts writers or interactively was also produced based on these primitive operations.

The example in Fig 2 shows a small script for the client, which is called smtcl. The script connects to a server named on the command line, finds out the identifiers of all of its components ³, and then gets the value of the operational state attribute (ifOperState) for each one. All of the management-specific sub-commands are prefixed with sysman.

The amount of code needed to provide this functionality was fairly small. The total amount of code required for the extension to Tcl was 3200 lines of C. The library of additional Tcl routines amounts to a further 1000 lines.

The main issues to address were in mapping the C-language data-types used for attributes in the management interface into appropriate Tcl structures and 'thread-safety'. DCE makes extensive use of multi-threading within a single address space and so some work was necessary in order to convert Tcl to use thread-safe library routines. In all, this required changes to less than 100 lines of code in the Tcl sources.

5.6 A graphical interface

Tcl has an accompanying graphical extension — Tk — which allows a Motif-like look-and-feel graphical user interface to be constructed from Tcl scripts. A further set of extensions known as Tix is also publicly available under the

same terms as Tcl and Tk, although from a different source [4]. Tix provides a wider range of graphical components.

```
#!/bin/sh
# This line restarts the script under smtcl \
  exec smtcl "$0" ${1+ "$@"}
#
# Simple server status display script.
#
# SCCS ID @(#)svrstatus.tcl          1.3 96/03/06

# Parse command-line options

if {[length $argv] != 1} {
  error "$argv0: Missing CDS name or string binding"
}

# Fill array with textual names of operational states

foreach name [array names SM_OPSTATE] {
  set statename($SM_OPSTATE ($name)) $name
}
# Connect to the server

sysman connect $argv

# Show details about each interface.

foreach ifdesc [sysman iflist] {
  set name [lindex $ifdesc 0]
  set exporting [lindex $ifdesc 1]
  set id [lindex [lindex $ifdesc 2] 0]

  puts -nonewline [format "%-40.40s " $name]
  if {![catch "sysman get $id ifOperState" result]} {
    puts -nonewline [format "%-12.12s " $statename([lindex
      $result 2])]
  } {
    puts -nonewline "(No opstate) "
  }
  if {$exporting} {puts Exporting} {puts Non-exporting}
}
sysman disconnect
```

Fig 2 Example client script.

It is difficult to build a robust version of the management client which incorporates the windowing extensions directly because of thread-safety issues. However, it is very straightforward to 'wrap' the non-graphical interface with Tk/Tix scripts, particularly as the output from a Tcl-enabled subprocess can be used directly as input to Tk/Tix commands. One such graphical interface, has been built for the management client. Some screen-shots are shown in Fig 3 and Fig 4. It is a testament to the flexibility and power of the Tcl/Tk/Tix combination that the entire interface is only about 3500 lines of code.

6. Future directions

The management interface and associated code were developed over a period of about 12 months. The first servers incorporating this interface have now been successfully deployed into the live operational environment. Feedback from operations personnel has generally been good. The

² Use is made of DCE facilities so that the management interface for a server can be found implicitly through names of the 'normal' functional interfaces without needing its own entries in the DCE name-space.

³ For historical reasons components are referred to as 'interfaces', hence the sub-command that lists the component identifiers is iflist for 'interface list'.

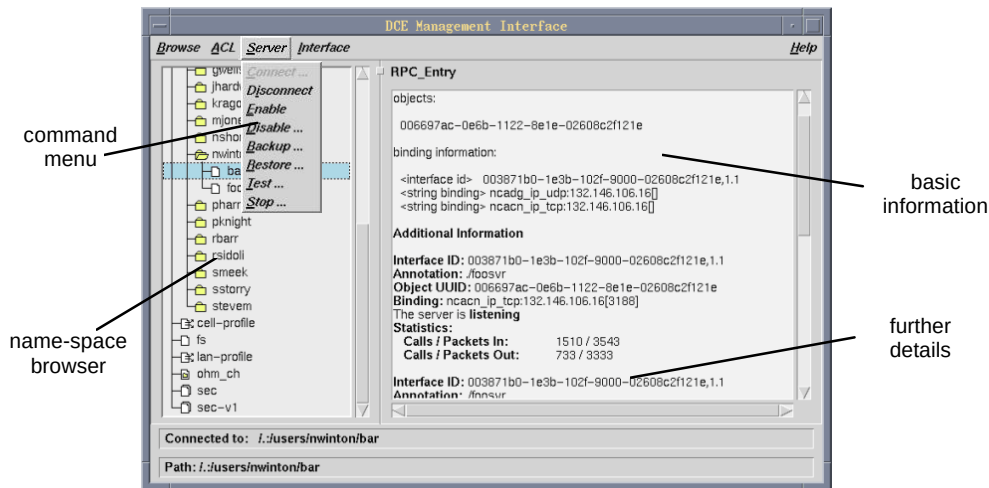


Fig 3 The GUI main screen.

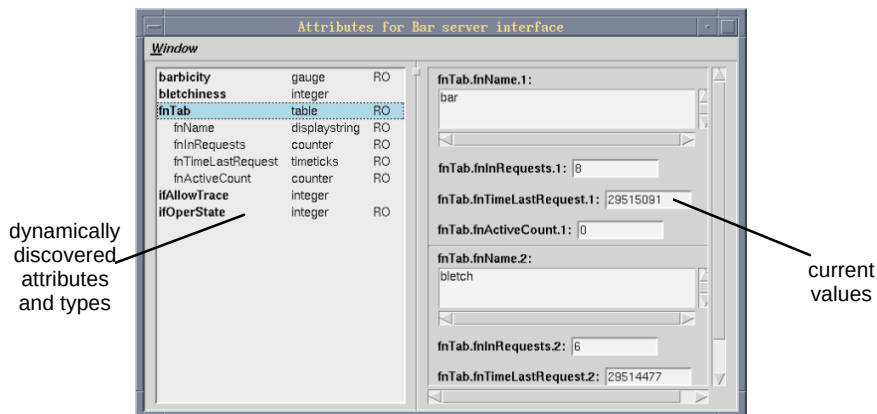


Fig 4 Dynamic component attribute display.

level of monitoring and control of these servers is in practice far greater than that for many non-distributed applications.

6.1 Buy versus build?

The intention was not to build an in-house solution, but instead to have been able to buy many of the capabilities 'off the shelf'. Even one year on, the capabilities provided exceed anything that could be purchased at the moment. However, there are some encouraging signs within the industry.

At the end of March 1996 the Desktop Management Task Force (DMTF) published version 2.0 of its Desktop Management Interface Specification (DMI) [5]. This document contains, among other things, a set of standard DCE interfaces which 'service providers' can support in order to allow them to be managed by standard clients. These interfaces provide broadly similar services to those in our management interface and in a broadly similar, SNMP-like style.

As yet there are few if any products that could be bought that would provide implementations of these interfaces for BT. Most of the DMTF effort has, thus far, been devoted to the management of standard hardware or software components, not user-written applications. One effort which may bear fruit in this area is Tivoli's Application Management Specification (AMS) [6]. This is based on the DMTF work but specifically sets out to tackle the issue of application management.

6.2 Further development

Further development of the interface and supporting code is not planned at present. Stability is more important than perfect functionality at the moment. Any future developments would be likely to simplify some aspects of the interface and to jettison a few historical artefacts of the design which cause a little confusion to users.

The intention, certainly, is to take the best ideas into any new tools or environment that can be bought in. Future work is likely to be additive and complementary rather than directly competitive.

7. Conclusions

Defining a standard management interface and providing an infrastructure that implements it 'for free' has allowed BT to build servers that have a very high degree of manageability. This is significantly better than anything that could be bought in at present.

The production of a generic, extensible client has allowed great flexibility both in integrating into the existing environment and in building additional functionality on to elementary services. The choice of Tcl has proved both easy and productive.

Not all of the management problems associated with distributed systems have been solved, but this work has contributed at least one significant part of the overall solution.

References

- 1 Case J D, Fedor M, Schoffstall M L and Davis C: 'RFC 1157: Simple network management protocol (SNMP)', (May 1990).
- 2 Rose M T and McCloghrie K: 'RFC 1155: Structure and identification of management information for TCP/IP-based Internets', (May 1990).
- 3 Ousterhout J: 'Tcl and the Tk toolkit', Addison-Wesley (1994).
- 4 <http://www.xpi.com/tix/>
- 5 'Desktop management interface specifications', Desktop Management Task Force Inc, Version 2.0 (March 1996).
- 6 <http://www.tivoli.com/AMS>



Neil Winton has an MA in Natural Sciences from Christ's College, Cambridge. He joined BT from university in 1984.

He has worked on a wide range of projects many of which have involved distributed system technologies and Unix. He has represented BT on several international committees, particularly concerned with systems management, and is the co-author of a book on distributed computing.

He was one of the main technical architects for the introduction of client/server technology into BT's operational support systems.